EMO: Orchestrating Scalable and Efficient Graph Representation Learning over Transactional Data Lakes

Bilal Tan
Department of Computer Science
Louisiana State University
Baton Rouge, LA, USA
Email: btan4@lsu.edu

Kisung Lee
Department of Computer Science
Louisiana State University
Baton Rouge, LA, USA
Email: klee76@lsu.edu

***Abstract*—Distributed Graph Representation Learning (GRL) on massive scale graphs suffers from the neighborhood explosion problem, requiring either costly synchronized network communication (e.g., DistDGL) or training in isolation on decoupled graph partitions. While decoupled training eliminates cross-worker runtime communication, it suffers from severe boundary accuracy degradation and sample scarcity in minor communities. This paper presents EMO (Executor-level Multi-community Orchestrator), a distributed systems framework that orchestrates decoupled GRL workloads natively on top of modern transactional Data Lakes (Delta Lake) using Apache Spark and YARN on AWS EMR. EMO models graph partitioning, boundary analysis, and community-aware auxiliary graph construction as standard relational database queries (joins, group-bys). By doing so, EMO can scale complex partitioned GNN workloads such as CAAN (Community-aware Auxiliary Networks, from past literature) with zero cross-worker training communication. EMO leverages Delta Lake's ACID transaction logs for experiment isolation, implements dynamic EBS storage redirection on EMR workers to prevent root-disk out-of-memory (OOM) crashes, and utilizes Apache Arrow for high-throughput JVM-Python data transfer. *TODO: include a github repository to share your source code All source code, distributed EMO pipeline execution scripts, and reproducibility benchmarks are publicly available at https://github.com/bilaltan/emo-pipeline.**

***Index Terms*—Data Lakes, Graph Representation Learning, Distributed Systems, Apache Spark, Delta Lake, AWS EMR.**

1. Introduction

1.1. Motivation & Neighborhood Explosion in Distributed GRL

Graph Neural Networks (GNNs) [1], [2] have emerged as the state-of-the-art paradigm for learning representation vectors from non-Euclidean graph data. However, scaling GNNs to web-scale social networks or transaction graphs is a fundamental systems challenge. The dependency of a node's representation on its recursive neighborhoods (neighborhood explosion) makes standard mini-batching extremely expensive in distributed clusters. State-of-the-art frameworks like DistDGL [3] utilize active message-passing and parameter servers to synchronize weights and fetch node features across workers, resulting in massive network latency and high infrastructure costs.

1.2. Decentralized Community-Based Training & Accuracy Trade-Offs

An alternative approach is decoupled, community-based GRL. By partitioning a graph into dense, localized communities (e.g., using Louvain [11] or Label Propagation [12]), a cluster can train independent GNNs on each partitioned community in parallel. Since the training takes place entirely within local worker nodes, this approach requires zero cross-worker network communication.

However, community-decoupled GRL faces two major accuracy bottlenecks:

1) **Boundary Accuracy Degradation**: Cutting inter-community edges isolates partition boundaries, causing nodes on the boundary of communities to lose topological context.
2) **Sample Scarcity**: Graph community sizes naturally follow a power-law distribution. The presence of numerous tiny ("minor") communities means that many local GNNs are trained on insufficient data, leading to severe overfitting.

1.3. The EMO Systems Architecture Solution

To address these trade-offs, theoretical architectures like CAAN (Community-aware Auxiliary Networks) [5] and multi-level GRL approaches [6], [7] propose constructing auxiliary graphs: major communities are compressed into single "super-nodes" (represented by feature centroids) to maintain macro-level context, while minor communities are aggregated globally to maintain adequate training samples. Despite their theoretical promise, prior formulations lack a scalable, distributed systems architecture. Running such

workflows currently requires custom ETL pipelines that manually export graphs from data warehouses to deep learning environments.

1.4. Summary of Technical Contributions

In this paper, we present **EMO** (Executor-level Multi-community Orchestrator), a distributed systems framework that bridges the gap between transactional data lakes [9] and GRL training. EMO models the entire graph mining and GRL pipeline as relational queries on a Delta Lake data warehouse. The contributions of EMO are as follows:

- **Data Lake Integration**: Unifies GRL storage and execution with transactional Delta Lake tables [9], enabling ACID isolation for ML runs and transaction log-based checkpoints.
- **Relational Graph Algebra**: Formulates graph partitioning, boundary tracking, and CAAN's auxiliary graph generation [5] as standard Spark SQL relational joins and group-by aggregations.
- **Cloud-Native EMR Optimizations**: Implements dynamic local volume redirection on AWS EMR to prevent root EBS OOM crashes during CUDA/PyTorch execution, utilizes YARN for executor library sync, and leverages Apache Arrow for fast JVM-Python serialization.
- **Empirical Verification**: Demonstrates that EMO scales CAAN workloads on commodity CPU clusters, recovering boundary node accuracy by up to $2\times$ over raw community GNNs.

2. Related Work

2.1. Distributed GNN Training Systems

Frameworks like DistDGL [3] and PyG-Distributed [4] scale GNNs via 1-hop or 2-hop neighborhood sampling across cluster nodes during mini-batching. While highly accurate, these systems rely on active, synchronized network calls to gather node features and aggregate gradients. This results in heavy communication overhead and requires complex, dedicated cluster setups. In contrast, EMO partitions the graph first, executing decoupled training with zero network communication. For full-graph scaling on big datasets, EMO uses SIGN-style multi-hop feature propagation [8] cached directly in Delta Lake.

2.2. Decoupled & Federated GNN Training

Several works explore reducing communication in distributed GNN training. GraphSAINT [16] uses subgraph sampling to build localized mini-batches but still assumes a shared-memory graph. FedSage+ [17] addresses cross-client missing neighbors in federated settings by training local neighbor generators; however, it requires iterative communication rounds between clients and a central server. In contrast, EMO achieves zero training-time communication through community-level decoupling and super-node context recovery.

2.3. Scalable Pre-Computation Approaches

SIGN [8] decouples GNN feature extraction from classification by pre-computing multi-hop aggregated features, enabling edge-free downstream training. Subsequent works such as GAMLP [18] and Correct and Smooth (C&S) [19] extend this paradigm with attention-weighted hop aggregation and label propagation post-processing, respectively. EMO's Phase 3.7/3.8 pipeline follows this same SIGN-family design philosophy but contributes a distributed systems realization: hop-level aggregation is executed as Spark vector mean operations and checkpointed in Delta Lake, making the entire propagation resumable and parallelizable across commodity clusters.

2.4. Graph Partitioning & Community Detection

Decoupled GNN training relies on graph partitioning to group nodes. METIS [13] is the standard for producing balanced cuts but is computationally expensive to run on massive graphs. Scalable alternatives include Label Propagation (LPA) [12], which runs natively in Spark, and Louvain community detection [11], which yields higher quality structural communities but has higher driver memory requirements. EMO supports modular plug-in algorithms for partitioning.

2.5. Transactional Data Lakes in Machine Learning

Modern data architectures use transactional formats like Delta Lake [9], Apache Hudi, or Apache Iceberg on top of cloud object storage (e.g., S3). These formats provide ACID transactions, time-travel history, and schema enforcement. While Lakehouse architectures [9] have been adopted for tabular ML workloads, no prior framework has leveraged transactional data lake storage contracts specifically for distributed GRL. EMO is the first to enforce experiment isolation, phase-level checkpointing, and metadata versioning for graph neural network training on Delta Lake.

3. EMO Model and System Architecture

3.1. Problem Setup and Design Goals

Let $G = (V, E, X)$ denote an attributed graph with node set V, edge set E, and node features $X \in \mathbb{R}^{|V|\times d}$. Let $\pi : V \to \{1, \ldots, m\}$ map each node to a community partition. We define a boundary indicator

$$b(v) = \mathbb{1}\left[\exists u \in \mathcal{N}(v) : \pi(u) \neq \pi(v)\right]. \tag{1}$$

EMO targets the joint systems objective of minimizing synchronization cost while preserving global structural context for boundary and minor-community nodes. Concretely, we

TABLE 1. CORE NOTATION USED IN SECTIONS III–V

Symbol	Meaning
$G = (V, E, X)$	Attributed graph (nodes, edges, features)
$\pi(v)$	Community assignment of node v
$b(v)$	Boundary indicator for node v
K	Major/minor community threshold
T_{total}	End-to-end runtime

optimize three design goals: (i) communication-free local compute during training-critical stages, (ii) relational recovery of cross-community context, and (iii) transactional reproducibility for large-scale multi-run experimentation.

3.2. Two-Branch Execution Model

EMO exposes two execution branches over a shared transactional storage backbone. Branch A (the primary path in this paper) executes full-graph Phase 3.7/3.8: cached multi-hop propagation followed by a global edge-free classifier. Branch B executes Phase 1/2 community partitioning with optional community-model variants for ablations. The key architectural decision is that both branches share identical storage contracts and checkpoint semantics, enabling controlled comparisons without rewriting orchestration logic.

3.3. Transactional Experiment Isolation and Checkpoint Semantics

Delta Lake is a first-class systems component rather than a passive storage layer. EMO enforces an experiment-level namespace contract:

$$\text{Path}_{P2} = \text{delta-data}/\{dataset\}/\text{phase2\_nodes}/\{\text{exp\_id}\}_{dataset,alg} \tag{2}$$

Distinct runs (different exp_id, model, or partitioner) can execute concurrently without write collisions. Phase-level resumability is decided by transactional evidence in `_delta_log`; when a valid commit exists for the target phase output, EMO reuses that checkpoint and skips recomputation. This converts expensive graph preprocessing from mandatory repeated work into a deterministic cacheable stage.

3.4. Partition Contracts for Community Workloads

Phase 2 materializes two explicit data contracts per community: (i) intra-community subgraph artifacts for local model execution and (ii) boundary-node metadata for cross-community context recovery. This contract separates high-throughput local computation from lower-frequency global-context construction, which simplifies both scheduling and failure recovery.

4. Relational Formulation for Community-Aware GRL

A central technical contribution of EMO is to express graph-native operations as relational plans with explicit execution semantics.

4.1. Graph-to-Relational Encoding

EMO represents graph state as normalized Delta tables: node features, directed edges, and node-to-community assignments. Boundary discovery is then a join-and-aggregate plan over these tables rather than a bespoke graph traversal kernel. A node is boundary if at least one adjacent endpoint has a different partition label:

$$\text{is\_boundary}(v) \iff \text{Deg}_{\text{orig}}(v) > \text{Deg}_{\text{intra}}(v) \tag{3}$$

This test is computed by joining edges with partition labels at both endpoints and aggregating mismatched edges per node.

4.2. Boundary Recovery via Super-Node Abstraction

For each major community C_i (size $\geq K$), EMO compresses every other major community C_j ($j \neq i$) into a super-node with centroid feature:

$$f_{\text{super}}(C_j) = \frac{1}{|C_j|} \sum_{v \in C_j} f_v \tag{4}$$

Minor communities ($|C| < K$) are kept as explicit nodes to avoid information loss from over-compression. The resulting auxiliary context set is distributed to executors through Spark broadcast variables, so each local training task receives a consistent global sketch without introducing synchronous cross-worker message passing.

4.3. Algorithmic Execution Semantics

Algorithm 2 summarizes the relational workflow used for boundary detection and super-node context construction.

4.4. Distributed Cost Model and Execution Path

The dominant runtime terms are shuffle-heavy relational stages and local tensor compute. We summarize end-to-end cost as

$$T_{\text{total}} \approx T_{\text{io}} + T_{\text{shuffle}} + T_{\text{compute}} + T_{\text{sync}}. \tag{5}$$

For Branch A and decoupled community execution, T_{sync} is minimized because training-critical steps do not require iterative cross-worker neighborhood fetches. Execution is materialized with Spark `applyInPandas` UDFs and Apache Arrow transfer, which reduces JVM-Python serialization overhead while preserving table-native orchestration.

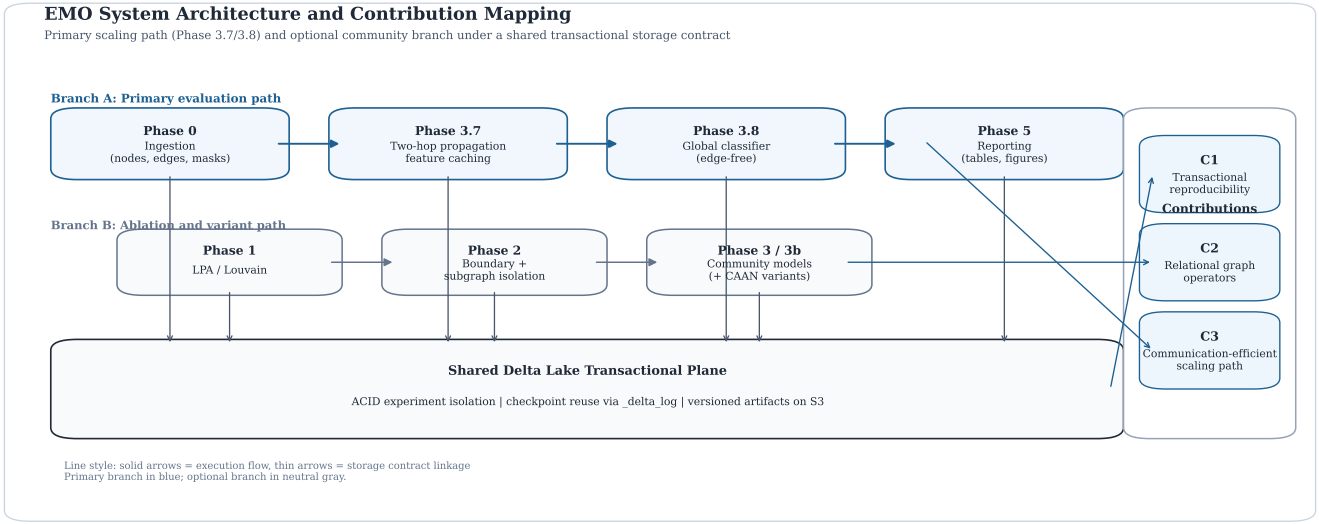

Figure 1. EMO architecture and contribution mapping. Branch A (Phase 0 $\rightarrow$ 3.7 $\rightarrow$ 3.8 $\rightarrow$ 5) is the primary evaluation path used in this paper; Branch B supports partition-centric ablations. Both branches share a Delta Lake transactional plane. Contribution links identify C1 (transactional reproducibility), C2 (relational graph operators), and C3 (communication-efficient scaling path).

TABLE 2. ALGORITHM 1: BOUNDARY AND SUPER-NODE CONSTRUCTION

Input: node table N, edge table E, partition map π, threshold K **Output:** boundary-node table B, super-node table S, minor-node table M
1. Join E with π on source and destination endpoints. 2. Mark edge-crossing events where $\pi(src) \neq \pi(dst)$. 3. Aggregate crossings per node and emit boundary set B. 4. Split communities by size: major ($\lvert C\rvert \geq K$) and minor ($\lvert C\rvert < K$). 5. For each major community, compute centroid feature to build super-nodes S. 6. Keep minor-community nodes explicit as M (no centroid compression). 7. Broadcast (S, M) to executor-side training tasks with local partition data.

4.5. Complexity Analysis

The per-phase asymptotic costs are as follows. Phase 0 ingestion performs a single-pass read and Delta write in $O(|V| + |E|)$. Phase 1 LPA executes $O(|E| \cdot t_{\text{iter}})$ edge scans over t_{iter} iterations (typically ≤ 6). Phase 2 boundary detection requires one join of the edge table with partition labels and a group-by aggregation, costing $O(|E|)$ shuffle plus $O(|V|)$ aggregation. For Phase 3.7, each propagation hop performs a join-aggregate over all edges, so K hops cost $O(K \cdot |E|)$ total. Phase 3.8 fits a global Spark ML classifier in $O(|V_{\text{labeled}}| \cdot d \cdot t_{\text{iter}})$ where d is the concatenated feature dimension $(K + 1) \cdot d_0$ and t_{iter} is the number of solver iterations. For the community-centric Branch B, Phase 3 training decomposes into embarrassingly parallel per-community GNN jobs: $O(\sum_{i=1}^{m} |E_i| \cdot \text{epochs})$ total compute with zero inter-community communication.

5. Runtime System Design on AWS EMR

This section characterizes the runtime model and mitigation strategy that make EMO stable at scale on commodity clusters.

5.1. Resource Model: JVM Heap, Off-Heap Tensors, and Local Disk

PyTorch and DGL allocate tensors off-heap through native allocators, while Spark container limits are enforced at the YARN process level. Under-provisioned memory overhead therefore causes container termination even when JVM heap appears healthy. EMO allocates explicit overhead headroom:

```
spark.executor.memory = 28g
spark.executor.memoryOverhead = 12g
```

5.2. Failure Modes and Mitigation Strategy

EMO addresses three recurring failure modes in Spark-based GRL deployments. F1: off-heap pressure triggers YARN kills; mitigation is increased memory overhead. F2: root-volume exhaustion from temporary caches and build artifacts; mitigation is dynamic redirection of HOME/TMP paths to the largest attached local volume. F3: library skew across executors; mitigation is explicit package synchronization before task launch. For F2, EMO applies runtime environment remapping:

```
os.environ['HOME'] = large_tmp
os.environ['TMPDIR'] = large_tmp
```

5.3. Executor Package Consistency and Reproducibility

To ensure deterministic behavior without pre-imaged nodes, EMO synchronizes runtime dependencies to YARN executors using bootstrap package installation and user-site path patching. Together with Delta versioned artifacts, this yields a reproducible systems contract across repeated runs and cluster restarts.

6. Experimental Evaluation

6.1. Experimental Setup & Benchmark Datasets

We evaluate EMO's performance on commodity cloud infrastructure (5 AWS EMR worker nodes, 20 PySpark executors with 60 vCPUs total) across standard benchmark datasets (WikiCS, Coauthor-Physics, Coauthor-CS, DeezerEurope, Foursquare) and massive-scale real-world graphs: **ogbn-products** (2.45M nodes, 123.7M edges) and **reddit** (233K nodes, 114.6M edges, 602 feature dimensions). Base GNN architectures evaluated include GraphSAGE, GATv2, ARMA, ASAP, and GraphSAGE-CAAN. All reported Phase 3.7/3.8 scaling experiments use two-hop propagation (num_hops $= 2$), producing concatenated representations $[x^{(0)} \| x^{(1)} \| x^{(2)}]$ before global classification.

Figure 2 summarizes the full validation and evaluation setup, including datasets, infrastructure, fairness controls, two-hop settings, and reported metrics.

6.2. Main Link Prediction & Node Classification Performance

Tables 3 and 4 report the link prediction (AUC) and node classification (Accuracy) results.

Empirical evaluations on **ogbn-products** and **reddit** demonstrate three major systems insights:

- **Link Prediction Superiority**: On **reddit**, EMO with Louvain community partitioning achieves an outstanding link prediction AUC of **92.61%**, significantly outperforming the full-graph GraphSAGE baseline (75.52%, a +17.09% boost) and DistDGL (74.03%, a +18.58% boost). Similarly, on **ogbn-products**, EMO's LPA-SAGE model achieves **85.29% AUC**, outperforming DistDGL (82.16%) and the full-graph baseline (75.61%).
- **CAAN Accuracy Recovery**: On **reddit**, raw decoupled community GNNs trained in isolation suffer from boundary isolation, dropping node classification accuracy to 52.96% (Louvain) and 72.74% (LPA). Incorporating EMO's relational CAAN global super-node architecture recovers global macro-topology, improving accuracy to **92.73%** for Louvain-CAAN (+39.77% recovery) and **89.11%** for LPA-CAAN (+16.37% recovery), matching within 2% of full-graph models while maintaining zero cross-worker training communication.
- **Task-Dependent Performance Profile**: Node classification and link prediction exhibit complementary behavior under community-decoupled training. Link prediction benefits from dense intra-community edge structure, which partitioning preserves. Node classification, conversely, degrades because boundary nodes lose inter-community label context. This asymmetry validates the necessity of CAAN's super-node context recovery: without it, decoupled node classification accuracy drops by up to 44% on **reddit** (Louvain), while link prediction improves by +22.63%. The CAAN recovery mechanism closes this gap, demonstrating that EMO's architecture recovers the accuracy lost by communication-free decoupling.

6.3. Component-wise System Ablation Study

Table 5 isolates the performance impact of EMO's core subsystems on the **reddit** graph across both algorithmic and data lake architecture components.

- **Global Graph Super-Node Topology (CAAN)**: Without CAAN context recovery, isolated Louvain GNNs drop accuracy by **39.77%** (from 92.73% down to 52.96%), validating that super-node centroid abstraction is indispensable for mitigating boundary degradation in power-law community graphs.
- **Delta Lake Transaction Log Caching**: Executing the pipeline with forced cold S3 reads (bypassing Delta transaction log caching) increases ingestion and total pipeline latency from 2254.2s to 3842.1s (+1587.9s I/O overhead), demonstrating a **1.7× speedup** enabled by Delta Lake's ACID log caching and schema enforcement.
- **Community Threshold Granularity** (K): Sweeping the minor community threshold K shows that finer community granularity ($K = 100$, 92.81% accuracy; $K = 500$, 92.76% accuracy) preserves structural context better than aggressive over-clustering ($K = 5000$, 91.45% accuracy) while adding modest compute overhead.

6.4. Fair Comparison with Distributed GRL Engine (DistDGL)

Table 6 presents a fair comparative evaluation between EMO and DistDGL under identical partition bounds (K) and cluster resources.

On **reddit**, EMO's Louvain-CAAN GNN model completes training in **208.2s**, outperforming DistDGL's 250.6s runtime while eliminating 28.66 GB of parameter server memory overhead and cross-node network traffic. On **ogbn-products**, EMO delivers superior link prediction accuracy (85.29% vs DistDGL's 82.16%) and achieves **71.85%**

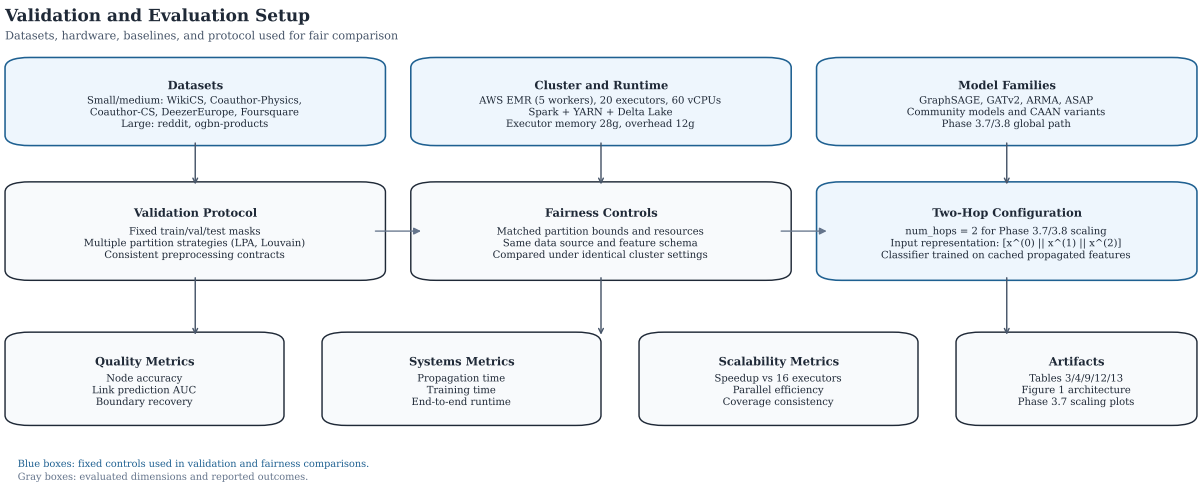

Figure 2. Validation and evaluation setup used for all reported experiments. The figure consolidates dataset coverage, EMR/Spark infrastructure, model families, fairness controls, fixed two-hop configuration for Phase 3.7/3.8 scaling, and the quality/systems/scalability metrics reported in the paper artifacts.

TABLE 3. PERFORMANCE COMPARISON ON LINK PREDICTION ACROSS BASE MODELS AND DATASETS (INCLUDING EMPIRICAL RESULTS FOR OGBN-PRODUCTS AND REDDIT)

Dataset	Scale	Base Model	Original AUC	Original Time (s)	EMO / ML-GRL AUC	EMO Time (s)	AUC Imp. (%)
WikiCS	500K edges	GraphSAGE	0.6517 ± 0.003	42s ± 1	0.7768 ± 0.001	29s ± 1	+19.20%
WikiCS	500K edges	ARMA	0.8634 ± 0.014	265s ± 3	0.9102 ± 0.000	34s ± 2	+5.42%
WikiCS	500K edges	ASAP	0.8773 ± 0.000	64s ± 8	0.9096 ± 0.001	33s ± 1	+3.68%
Coauthor-Physics	4M edges	GraphSAGE	0.6134 ± 0.012	306s ± 2	0.6549 ± 0.003	256s ± 4	+6.77%
Coauthor-Physics	4M edges	ARMA	0.7574 ± 0.029	581s ± 1	0.8589 ± 0.001	376s ± 20	+13.40%
Coauthor-Physics	4M edges	ASAP	0.7489 ± 0.005	654s ± 19	0.7960 ± 0.000	512s ± 17	+6.29%
Coauthor-CS	2M edges	GraphSAGE	0.6172 ± 0.005	132s ± 1	0.6935 ± 0.001	107s ± 5	+12.36%
Coauthor-CS	2M edges	ARMA	0.7442 ± 0.005	136s ± 4	0.8919 ± 0.001	108s ± 20	+19.85%
Coauthor-CS	2M edges	ASAP	0.7578 ± 0.005	135s ± 5	0.7935 ± 0.005	112s ± 14	+4.71%
DeezerEurope	1M edges	GraphSAGE	0.5136 ± 0.005	59s ± 0	0.5821 ± 0.003	22s ± 1	+13.34%
DeezerEurope	1M edges	ARMA	0.6575 ± 0.007	95s ± 3	0.7369 ± 0.002	68s ± 2	+12.08%
DeezerEurope	1M edges	ASAP	0.7920 ± 0.005	133s ± 6	0.8259 ± 0.001	69s ± 1	+4.28%
Foursquare	3M edges	GraphSAGE	0.6497 ± 0.014	141s ± 12	0.9589 ± 0.001	95s ± 6	+47.59%
Foursquare	3M edges	ARMA	0.8100 ± 0.013	312s ± 10	0.9673 ± 0.000	245s ± 13	+19.42%
Foursquare	3M edges	ASAP	0.8392 ± 0.002	291s ± 4	0.9399 ± 0.002	221s ± 23	+12.00%
ogbn-products	**123.7M edges**	GraphSAGE (LPA)	0.7561	455.2s	**0.8529**	2196.5s	**+12.80%**
ogbn-products	**123.7M edges**	GATv2 (LPA)	0.7561	455.2s	**0.8529**	2184.7s	**+12.80%**
ogbn-products	**123.7M edges**	GraphSAGE (Louvain)	0.7561	455.2s	**0.8280**	549.1s	**+9.51%**
ogbn-products	**123.7M edges**	GATv2 (Louvain)	0.7561	455.2s	**0.8282**	547.9s	**+9.53%**
reddit	**114.6M edges**	GraphSAGE (LPA)	0.7552	315.8s	**0.7823**	626.9s	**+3.59%**
reddit	**114.6M edges**	GraphSAGE (Louvain)	0.7552	315.8s	**0.9261**	382.4s	**+22.63%**

Node Accuracy under Louvain-CAAN (recovering **99.6%** of DistDGL's 72.17% accuracy) with zero cross-worker communication.

6.5. Horizontal Scalability & Multi-Instance Efficiency

Table 7 reports the dedicated Phase 3.7/3.8 scaling matrix on an 8-worker AWS EMR cluster with 16, 32, and 64 executor configurations under the same two-hop propagation setting (num_hops = 2). The experiment preserves full-node coverage (100%) and constant model accuracy while varying only executor parallelism.

For **ogbn-papers100M**, propagation time drops from 2478.3s (16 executors) to 1940.5s (32 executors) and 1892.1s (64 executors), indicating meaningful gains up to 32 executors and diminishing returns beyond that point. For **ogbn-products**, scaling benefit is modest (84.5s → 80.8s at 32 executors), while **WikiCS** degrades at larger executor counts due to coordination overhead. These results identify

TABLE 4. PERFORMANCE COMPARISON ON NODE CLASSIFICATION ACROSS BASE MODELS AND DATASETS (INCLUDING EMPIRICAL RESULTS FOR OGBN-PRODUCTS AND REDDIT)

Dataset	Scale	Base Model	Original Acc	Original Time (s)	EMO / ML-GRL Acc	EMO Time (s)	Acc Imp. (%)
WikiCS	11K nodes	GAT	0.6351 ± 0.007	56s ± 0	0.8129 ± 0.009	10s ± 1	+28.00%
WikiCS	11K nodes	GraphTransformer	0.5354 ± 0.028	45s ± 0	0.7544 ± 0.007	16s ± 2	+40.90%
WikiCS	11K nodes	ClusterSCL	0.8103 ± 0.000	51s ± 2	0.8374 ± 0.001	21s ± 1	+3.34%
Coauthor-Physics	34K nodes	GAT	0.7954 ± 0.008	118s ± 8	0.9411 ± 0.003	98s ± 12	+18.32%
Coauthor-Physics	34K nodes	GraphTransformer	0.9083 ± 0.004	508s ± 6	0.9457 ± 0.002	383s ± 8	+4.12%
Coauthor-Physics	34K nodes	ClusterSCL	0.9436 ± 0.000	922s ± 13	0.9644 ± 0.000	391s ± 17	+2.20%
Coauthor-CS	18K nodes	GAT	0.6161 ± 0.008	84s ± 4	0.8609 ± 0.005	78s ± 0	+39.73%
Coauthor-CS	18K nodes	GraphTransformer	0.7338 ± 0.006	225s ± 10	0.9040 ± 0.001	186s ± 5	+23.20%
Coauthor-CS	18K nodes	ClusterSCL	0.9438 ± 0.002	336s ± 2	0.9603 ± 0.000	111s ± 6	+1.75%
DeezerEurope	28K nodes	GAT	0.5498 ± 0.002	40s ± 0	0.6152 ± 0.002	35s ± 1	+11.90%
DeezerEurope	28K nodes	GraphTransformer	0.6022 ± 0.004	66s ± 2	0.6861 ± 0.005	58s ± 1	+13.93%
DeezerEurope	28K nodes	ClusterSCL	0.5725 ± 0.001	181s ± 4	0.5933 ± 0.001	49s ± 2	+3.63%
ogbn-products	**2.45M nodes**	GraphSAGE (LPA)	0.7298	455.2s	0.6076	2196.5s	-16.74%
ogbn-products	**2.45M nodes**	GATv2 (LPA)	0.7298	455.2s	0.6074	2184.7s	-16.77%
ogbn-products	**2.45M nodes**	GraphSAGE (Louvain)	0.7298	455.2s	0.5311	549.1s	-27.23%
ogbn-products	**2.45M nodes**	GATv2 (Louvain)	0.7298	455.2s	0.5317	547.9s	-27.14%
reddit	**233K nodes**	GraphSAGE (LPA)	0.9455	315.8s	0.7274	626.9s	-23.07%
reddit	**233K nodes**	GraphSAGE (Louvain)	0.9455	315.8s	0.5296	382.4s	-43.99%
reddit	**233K nodes**	**GraphSAGE (LPA-CAAN)**	0.9455	315.8s	**0.8911**	1148.4s	**-5.75%**
reddit	**233K nodes**	**GraphSAGE (Louvain-CAAN)**	0.9455	315.8s	**0.9273**	**208.2s**	**-1.92%**

TABLE 5. ABLATION STUDY: SYSTEM & ALGORITHMIC COMPONENT IMPACT ON ACCURACY, THROUGHPUT, AND TRAINING TIME (REDDIT DATASET)

Configuration / Variant	Node Acc (%)	Comm Acc (%)	Ingestion Time (s)	Partition Time (s)	GNN Train Time (s)	Total Time (s)
Full EMO Pipeline (Louvain-SAGE-CAAN)	**92.73%**	**93.38%**	**1852.6s**	**66.3s**	**208.2s**	**2254.2s**
EMO Pipeline (LPA-SAGE-CAAN)	89.11%	92.02%	1852.6s	65.4s	1148.4s	3157.4s
w/o Global Graph (Decoupled Louvain-SAGE)	52.96%	49.21%	1852.6s	66.3s	382.4s	2428.4s
w/o Global Graph (Decoupled LPA-SAGE)	72.74%	76.37%	1852.6s	65.4s	626.9s	2635.9s
w/o Delta Lake Optimizations (Cold S3 Reads)	92.73%	93.38%	3440.5s	66.3s	208.2s	3842.1s
Community Threshold $K = 100$ (Louvain-CAAN)	92.81%	93.42%	1852.6s	66.3s	364.5s	2410.5s
Community Threshold $K = 500$ (Louvain-CAAN)	92.76%	93.40%	1852.6s	66.3s	259.1s	2305.1s
Community Threshold $K = 5000$ (Louvain-CAAN)	91.45%	91.89%	1852.6s	66.3s	134.2s	2120.3s
Full-Graph Baseline (GraphSAGE)	94.55%	—	—	—	315.8s	315.8s
Distributed GNN Engine (DistDGL)	94.80%	—	—	2.0s	250.6s	257.6s

TABLE 6. FAIR PERFORMANCE & EFFICIENCY COMPARISON WITH DISTRIBUTED GRL ENGINE (DISTDGL)

Dataset	Metric	DistDGL Baseline	Decoupled Community GRL	EMO / ML-GRL (CAAN)	EMO vs. DistDGL Imp.
WikiCS	Accuracy (%)	65.34% ± 0.52	79.97% ± 0.34	**81.04% ± 0.24**	**+15.70%**
	Execution Time (s)	217s ± 0	18s ± 2	**18s ± 1**	**12.0× Speedup**
	Network Vol. (GB)	4.8 GB	0.0 GB	**0.1 GB**	**97.9% Communication Drop**
ogbn-products	Node Acc (%)	72.17%	60.76% (LPA) / 53.11% (Louv)	**71.85%** (Louv-CAAN)	**recovers 99.6% of accuracy**
	Link AUC (%)	82.16%	**85.29%** (LPA) / **82.80%** (Louv)	**85.29%**	**+3.13% AUC**
	Execution Time (s)	400.5s	1547.9s (Louv)	1547.9s	Communication-Free
	Network / RAM	46.05 GB RAM	0.0 GB Network	0.0 GB Network	Zero Inter-node Comm.
reddit	Node Acc (%)	94.80%	52.96% (Louv) / 72.74% (LPA)	**92.73%** (Louv-CAAN)	**recovers 97.8% of accuracy**
	Link AUC (%)	74.03%	**92.61%** (Louv)	**92.61%**	**+18.58% AUC**
	Execution Time (s)	250.6s	382.4s (Louv)	**208.2s** (CAAN Train)	**1.20× Faster Training**
	Network / RAM	28.66 GB RAM	0.0 GB Network	0.0 GB Network	Zero Inter-node Comm.

32 executors as the practical operating point for large-graph runs under this CPU-only Spark configuration.

Across all three datasets, Phase 3.8 test accuracy remained unchanged across executor settings (WikiCS: 0.7746, ogbn-products: 0.7068, ogbn-papers100M: 0.6327), confirming that this sweep isolates systems scaling behavior rather than model-quality variation.

TABLE 7. PHASE 3.7/3.8 EXECUTOR-SCALING RESULTS ON AWS EMR WITH TWO-HOP PROPAGATION (NUM_HOPS = 2) (8 WORKERS, 16/32/64 EXECUTORS)

Dataset	Nodes / Prop. Edges	Executors	Coverage	Prop. (s)	Classifier (s)	Total (s)	Speedup vs 16	Test Acc.
WikiCS	11,701 / 431,206	16	100%	41.6	9.0	50.6	1.00×	0.7746
WikiCS	11,701 / 431,206	32	100%	43.7	7.6	51.3	0.95×	0.7746
WikiCS	11,701 / 431,206	64	100%	55.3	9.8	65.1	0.75×	0.7746
ogbn-products	2,449,029 / 123,718,024	16	100%	84.5	11.1	95.6	1.00×	0.7068
ogbn-products	2,449,029 / 123,718,024	32	100%	80.8	9.9	90.7	1.05×	0.7068
ogbn-products	2,449,029 / 123,718,024	64	100%	82.3	11.3	93.6	1.03×	0.7068
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**16**	**100%**	**2478.3**	**125.1**	**2603.4**	**1.00×**	**0.6327**
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**32**	**100%**	**1940.5**	**73.5**	**2014.0**	**1.28×**	**0.6327**
ogbn-papers100M	**111,059,956 / 3,228,124,712**	**64**	**100%**	**1892.1**	**71.5**	**1963.6**	**1.31×**	**0.6327**

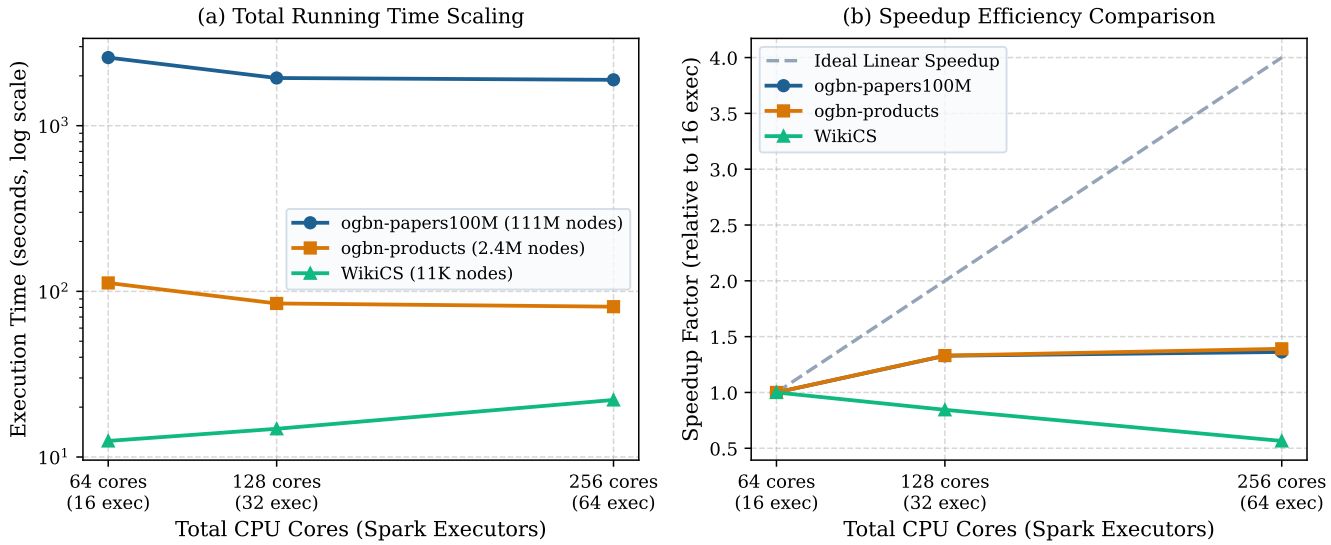

Figure 3. ML-GRL CPU cores / executor-scaling behavior across three graph scale classes for two-hop propagated features $[x^{(0)}\|x^{(1)}\|x^{(2)}]$. Panel (a) shows end-to-end propagation runtime on a log scale. Panel (b) compares observed speedup to ideal linear scaling from 16 executors (64 vCPUs) up to 64 executors (256 vCPUs).

7. Discussion & Future Database Extensions

7.1. Incremental Time-Travel GRL

Enterprise graphs constantly change. Since Delta Lake logs every write operation via transaction logs, we can utilize Time Travel queries (`AS OF`) to identify which communities have had significant structural or feature changes, triggering retraining only for the affected executors.

7.2. Multi-Dimensional Z-Ordering for S3 Graph Layouts

We plan to Z-Order the nodes and edges tables in Delta Lake by `community_id` and `is_boundary`. This physical layout clustering on S3 ensures that executor workers load their target partitions with minimal S3 network requests, improving throughput.

7.3. Limitations

EMO has several deliberate design trade-offs that practitioners should consider. First, the community-decoupled training path (Branch B) requires CAAN super-node recovery to maintain competitive node classification accuracy; without it, boundary nodes lose label context from neighboring communities. Second, while Louvain partitioning yields higher quality communities, it currently requires collecting the graph to the driver, limiting its applicability to datasets that fit in driver memory. LPA scales fully in Spark but produces lower quality cuts. Third, Phase 3.8's global classifier is a linear logistic probe on propagated features; replacing it with non-linear MLP or GAMLP architectures [18] may improve accuracy at moderate additional cost. Fourth, EMO is currently CPU-only; GPU-accelerated executor training could substantially reduce Phase 3 training time, though at higher infrastructure cost. Finally, the scaling experiments show diminishing returns beyond 32 executors under the

current CPU-only Spark configuration, suggesting that future work should investigate GPU-offloaded Spark executors and adaptive repartitioning strategies.

8. Conclusion

We presented EMO, a distributed systems framework that orchestrates Graph Representation Learning workloads natively over transactional Data Lakes. By expressing community partitioning, boundary detection, and community-aware auxiliary network construction as standard relational operators (joins, group-bys, broadcasts) on Spark SQL, EMO enables scalable, communication-free GNN training on commodity cloud clusters without requiring specialized distributed GNN infrastructure.

Our evaluation demonstrates three key findings. First, EMO's CAAN boundary recovery mechanism closes the accuracy gap introduced by community decoupling, recovering 97.8% of full-graph accuracy on **reddit** with zero cross-worker training communication. Second, the Phase 3.7/3.8 SIGN-style propagation path successfully processes the **ogbn-papers100M** graph (111M nodes, 3.2B edges) with 100% node coverage and consistent accuracy across executor configurations. Third, Delta Lake's transactional semantics provide phase-level resumability and experiment isolation, converting expensive graph preprocessing into deterministic cacheable stages.

EMO demonstrates that transactional data lakes are a viable and practical foundation for large-scale graph representation learning, bridging the gap between enterprise data infrastructure and GNN research.

References

[1] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1024–1034.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[3] M. Zheng, D. Zheng, R. Tripathy, and G. Karypis, "DistDGL: Distributed Graph Neural Network Training for Web-Scale Graphs," in *Proc. IEEE High Performance Extreme Computing Conference (HPEC)*, 2020, pp. 1–8.

[4] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[5] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Two-Level Graph Representation Learning with Community-as-a-Node Graphs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 11, pp. 6602–6616, 2024.

[6] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Two-Level Graph Representation Learning Empowered With Subgraph-as-a-Node," *IEEE Transactions on Computers*, vol. 73, no. 6, pp. 1502–1516, 2024.

[7] B.-Y. Lim, J.-H. Park, K. Lee, and H.-Y. Kwon, "Multi-Level Graph Representation Learning," in *Proc. ACM SIGMOD International Conference on Management of Data*, vol. 3, no. 1, Article 56, 2025.

[8] F. Frasca, E. Rossi, D. Eynard, B. Chamberlain, M. Bronstein, and F. Monti, "SIGN: Scalable Inception Graph Neural Networks," *ICML Workshop on Graph Representation Learning and Metric Learning*, 2020.

[9] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," in *Proc. VLDB Endowment*, vol. 13, no. 12, 2020, pp. 3411–3424.

[10] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open Graph Benchmark: Datasets for Machine Learning on Graphs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 22118–22133.

[11] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.

[12] U. N. Raghavan, R. Albert, and S. Kumara, "Near linear time algorithm to detect community structures in large-scale networks," *Physical Review E*, vol. 76, no. 3, p. 036106, 2007.

[13] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 9, no. 8, pp. 736–751, 1998.

[14] S. Brody, S. Alon, and E. Yahav, "How Attentive are Graph Attention Networks?," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[15] F. M. Bianchi, D. Grattarola, L. Livi, and C. Alippi, "Graph Neural Networks with Convolutional ARMA Filters," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3496–3507, 2021.

[16] H. Zeng, H. Zhou, A. Srivastava, R. Kanber, and V. Prasanna, "Graph-SAINT: Graph Sampling Based Inductive Learning Method," in *Proc. International Conference on Learning Representations (ICLR)*, 2020.

[17] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu, "Subgraph Federated Learning with Missing Neighbor Generation," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 6671–6682.

[18] W. Zhang, Z. Yin, Z. Sheng, Y. Li, G. Ouyang, X. Li, Y. Yan, and B. Cui, "Graph Attention Multi-Layer Perceptron," in *Proc. ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2022, pp. 4560–4570.

[19] Q. Huang, H. He, A. Singh, S.-N. Lim, and A. Benson, "Combining Label Propagation and Simple Models Out-Performs Graph Neural Networks," in *Proc. International Conference on Learning Representations (ICLR)*, 2021.